The C++ you need (1 of 2)

Everything you need to write the code in this course. Stuck? Look here first, then ask your group, then an instructor.

1 The shape of every program

INITIALIZATION: the top of the file. Libraries, settings and variables. Read once, before anything runs.

```
#include <Adafruit_NeoPixel.h>   // pull in a library
const int LED_PIN = 2;           // a setting: never changes
int loopCount = 0;               // a variable: can change
```

setup() runs ONCE, at power-up or reset. Get the hardware ready.

```
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
}
```

loop() runs over and over, as fast as it can, until the power goes off.

```
void loop() {
  loopCount = loopCount + 1;
  digitalWrite(LED_PIN, HIGH);
  delay(1000);
}
```

The rules that catch everybody. Every statement ends with `;`. Every `{` has a matching `}`. `//` starts a comment the computer ignores. Capitals matter: `Serial` is not `serial`. A name is one word - letters, digits and _ - and cannot start with a digit.

2 Variables: named boxes that hold a value

Type	Holds	Example
`int`	A whole number, up to about 2 billion either way	`int speed = 200;`
`float`	A number with a decimal point	`float volts = 16.8;`
`bool`	Only true or false	`bool lightsOn = true;`
`unsigned long`	A big whole number, never negative - for millis()	`unsigned long lastBlink = 0;`
`uint32_t`	A NeoPixel color, packed into one number	`uint32_t red = strip.Color(255, 0, 0);`
`const`	In front of any type: it can never change	`const int MOTOR_MAX = 255;`
`uint8_t x[6]`	A list of six numbers, 0 to 255	`const uint8_t MY_CONTROLLER[6] = { 0x58, 0x2F, 0x40, 0x11, 0x22, 0x33 };`

Make a variable ONCE, with its type; after that use the name on its own: `speed = 150;` A variable made inside `{ }` disappears at the `}`. If loop() must remember it from one pass to the next, make it at the top of the file.

3 Math, and the traps in it

Arithmetic. Whole numbers give whole answers: the fraction is thrown away, never rounded.

```
int total    = 7 + 2;     // 9
int half     = 7 / 2;     // 3, NOT 3.5
float exact  = 7.0 / 2;   // 3.5 - one decimal point does it
int leftOver = 7 % 2;     // 1, the remainder
int none     = 9 / 10;    // 0, and it does not round up
```

Changing a variable

```
count = count + 1;   count++;     // both add one
count += 5;          count--;     // add five, take one away
```

Multiply BEFORE you divide, or it comes out 0.

```
int percent = duty * 100 / 255;   // duty 128 gives 50
int wrong   = duty / 255 * 100;   // duty 128 gives 0
```

Handy functions

```
int size  = abs(-40);                  // 40: no sign
int safe  = constrain(300, -255, 255); // 255: kept in range
int speed = map(255, 0, 511, 0, 255);  // 127: proportion
int small = min(3, 8);                 // 3  (and max() gives the larger)
int pick  = random(0, 32);             // 0 to 31: the top is left out
```

4 Comparing, and combining tests

Comparisons come out true or false. TWO = signs to compare: one = SETS the value instead.

```
a == b  equal          a != b  not equal
a <  b  less than      a <= b  less than or equal
a >  b  more than      a >= b  more than or equal
if (speed = 0)   // WRONG: sets speed to 0, never compares
```

Combine tests: AND, OR, NOT.

```
if (speed > 0 && lightsOn) { }        // && both must be true
if (stickX < -20 || stickX > 20) { }  // || either one will do
if (!lightsOn) { }                    // !  flips true and false
```

The C++ you need (2 of 2)

5 Making decisions: if, else if, else

if runs its block only when the test is true; **else** when it is not - never both. **else if** makes a chain: the FIRST true test wins, so put the narrowest test first.
`test ? a : b` is a quick either/or for one value: a if the test is true, b if not.

```
if (duty > 200) {
  Serial.println("fast");
} else if (duty > 60) {
  Serial.println("moving");
} else {
  Serial.println("stopped");
}
int forwardDuty = (speed > 0) ? speed : 0;
Serial.println(lightsOn ? "Lights ON" : "Lights OFF");
```

6 Loops: doing something many times

for a known number of times: start here; keep going while this is true; change this each time round.

```
for (int i = 0; i < 32; i++) {
  strip.setPixelColor(i, strip.Color(0, 0, 255));
}
for (int i = 15; i >= 0; i--)  { strip.setPixelColor(i, 0); }  // down
for (int i = 0; i < 32; i += 2) { strip.setPixelColor(i, 0); }  // in twos
```

A run of LEDs: use `<=` when the last number counts, or you miss it - off by one.

```
for (int i = FRONT_FIRST; i <= FRONT_LAST; i++) {
  strip.setPixelColor(i, strip.Color(255, 255, 255));
}
```

while goes on as long as the test stays true, and might not run at all. `break;` leaves any loop early.

```
while (countdown > 0) {   // countdown is an int
  countdown--;
}
```

7 Functions: give a job a name, then call it

void means it hands nothing back. **Parameters** pass information in. Write it outside every other function, then **call it** from loop().

```
void lightRange(int first, int last, uint32_t color) {
  for (int i = first; i <= last; i++) {
    strip.setPixelColor(i, color);
  }
}
lightRange(0, 15, strip.Color(255, 0, 0));   // call it: front bar red
```

return hands a value back: its type goes where void was.

```
int percentOf(int duty) {
  return duty * 100 / 255;      // percentOf(128) is 50
}
```

8 The Serial Monitor: how the program talks to you

Tools > Serial Monitor. Nonsense characters: the baud box is not 115200.
Stuck? Print the numbers you are working from.

```
Serial.begin(115200);         // in setup()
Serial.print("Speed: ");       // print stays on the line
Serial.println(speed);        // println ends it
Serial.println(volts, 2);     // two decimal places
Serial.print("\t");            // a tab, to line up columns
```

9 Time: delay() stops everything, millis() does not

delay(1000) waits a second and NOTHING else happens, so never in a vehicle program. **millis()** is how long the board has been on, in ms. Check it every pass; act when enough time has gone by. Every job gets its OWN last-time variable.

```
unsigned long lastBlink = 0;
void loop() {
  if (millis() - lastBlink >= 500) {      // every half second
    lastBlink = millis();
    blinkOn = !blinkOn;       // ...do the job...
  }                           // everything else keeps running
}
```

10 Named states, lists, and reading sensors

enum names the states something can be in. An **array** is a list: `[i]` picks one, counting from 0. **Sensors** for the projects are read like the last two lines.

```
enum LightPattern { LIGHTS_STOPPED, LIGHTS_FORWARD, LIGHTS_REVERSE };
LightPattern pattern = LIGHTS_STOPPED;       // one named state
const int CORNERS[] = { 0, 15, 16, 31 };     // CORNERS[0] is 0
const int HOW_MANY = sizeof(CORNERS) / sizeof(CORNERS[0]);   // 4
int millivolts = analogReadMilliVolts(34);   // 0 to about 3300
long echo = pulseIn(ECHO_PIN, HIGH, 30000);  // microseconds, 0 if none
```

This vehicle (1 of 2)

11 The pins this vehicle uses

What	GPIO pins	Notes
Left motors	front 12, 13 rear 18, 19	Two pins each: the first drives forward,
Right motors	front 22, 23 rear 16, 17	the second reverse
The 32 LEDs	5	One data wire drives all of them
Servos 1 to 4	25, 26, 27, 14	On the main board. Breadboard LED: 2
I2C sensors	SDA 32, SCL 33	GPIO 34 to 39 are INPUT only

12 Motors

Set each pin up once, in setup().

```
ledcSetup(FRONT_LEFT_CH_A, 20000, 8);   // channel, 20 kHz, 8 bits
ledcAttachPin(FRONT_LEFT_PIN_A, FRONT_LEFT_CH_A);
```

Duty 0 to 255 is the power. A for forward, B for reverse; both 0 coasts.

```
ledcWrite(FRONT_LEFT_CH_A, 200);   // forward, about 78%
ledcWrite(FRONT_LEFT_CH_B, 0);     // B off
```

drive(left, right) does all four motors: -255 to +255, minus is reverse.

```
drive(200, 200);    // straight ahead
drive(150, -150);   // spin right on the spot
drive(0, 0);        // stop
```

Tank drive: one stick, two sides. constrain() stops 200 + 60 asking for 260.

```
int leftSpeed  = constrain(forward + turn, -MOTOR_MAX, MOTOR_MAX);
int rightSpeed = constrain(forward - turn, -MOTOR_MAX, MOTOR_MAX);
drive(leftSpeed, rightSpeed);
```

A wheel turns the wrong way? Swap that motor's two pin numbers in the sketch - never rewire. **Wheels off the ground** the first time you upload any change that drives.

13 The 32 LEDs (Adafruit NeoPixel)

Make the strip once, at the top of the file.

```
#include <Adafruit_NeoPixel.h>
const int LED_PIN   = 5;
const int LED_COUNT = 32;
Adafruit_NeoPixel strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
```

In setup()

```
strip.begin();
strip.setBrightness(60);   // 0 to 255, set once
```

Paint, then show. These only change memory: NOTHING lights up until `strip.show()`.

```
strip.setPixelColor(4, strip.Color(255, 0, 0));   // one LED
strip.fill(strip.Color(0, 255, 0));               // all of them
strip.clear();                                    // all off
strip.show();                                     // now send it!
```

Rainbows use a HUE, 0 to 65535, round the color wheel.

```
strip.setPixelColor(0, strip.gamma32(strip.ColorHSV(21845)));
int howMany = strip.numPixels();   // 32
```

The LED map. Front bar 0 (left) to 15 (right); the rear bar runs back, 16 (right) to 31 (left). The LED behind front LED p is 31 - p. LEFT side: 0-7 and 24-31. RIGHT side: 8-15 and 16-23. Corners: 0, 15, 16, 31.

	LEFT side of the vehicle								RIGHT side of the vehicle							
FRONT	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
REAR	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16

Colors: each one is strip.Color(red, green, blue)

Red	`Color(255, 0, 0)`	Yellow	`Color(255, 255, 0)`	Purple	`Color(128, 0, 255)`
Green	`Color(0, 255, 0)`	Cyan	`Color(0, 255, 255)`	White	`Color(255, 255, 255)`
Blue	`Color(0, 0, 255)`	Magenta	`Color(255, 0, 255)`	Warm white	`Color(255, 160, 80)`
Orange	`Color(255, 100, 0)`	Pink	`Color(255, 100, 100)`	Off	`Color(0, 0, 0)`

Power: each LED at full white draws about 60 mA, so all 32 draw nearly 2 amps. Red lights one of the three LEDs inside a pixel; white lights all three, and costs three times as much.

This vehicle (2 of 2)

14 The controller

At the top: the library, and YOUR controller's address - paste the line l3a_controller_check prints.

```
#include <Bluepad32.h>
const uint8_t MY_CONTROLLER[6] = { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };
ControllerPtr myController = nullptr;
```

Setting up takes two callbacks and an allowlist: copy them from l3c_tank_drive.

```
BP32.update();   // the first line of loop(), every time
```

Sticks: -512 to +511. Pushing UP gives a NEGATIVE number.

```
int leftX  = myController->axisX();    // left and right
int leftY  = myController->axisY();    // UP is negative
int rightX = myController->axisRX();   // axisRY() for up and down
```

Buttons are named by POSITION, not by the letter printed on the pad. True while held down.

```
a() bottom (B)   b() right (A)   x() left (Y)   y() top (X)
l1()  r1()  l2()  r2()   shoulders and triggers
thumbL()  thumbR()       click a stick in
bool upPressed = myController->dpad() & DPAD_UP;
```

15 The patterns the full program is built from

FAILSAFE. No controller, no driving: the very first lines of loop().

```
if (myController == nullptr || !myController->isConnected()) {
  drive(0, 0);
  return;      // skip the rest of loop()
}
```

DEADZONE AND MAP. Nearly centered counts as centered; the rest of the stick is stretched across the whole speed range.

```
int stickToSpeed(int stickValue, int maxSpeed) {
  if (abs(stickValue) < STICK_DEADZONE) {
    return 0;
  }
  int size = map(abs(stickValue), STICK_DEADZONE, STICK_MAX, MOTOR_MIN, maxSpeed);
  size = constrain(size, MOTOR_MIN, maxSpeed);
  return (stickValue > 0) ? size : -size;
}
```

EDGE DETECTION. One press, one action, however long the finger stays down. One wasDown per button.

```
bool justPressed(bool isDown, bool &wasDown) {
  bool isNewPress = isDown && !wasDown;
  wasDown = isDown;
  return isNewPress;
}
```

TOGGLE on each new press, and **DIRTY FLAG:** redraw only when something changed.

```
if (justPressed(myController->x(), lightsWasDown)) {
  lightsOn = !lightsOn;
  lightsChanged = true;
}
if (lightsChanged) {
  lightsChanged = false;
  showDrivingLights();
}
```

16 When it will not compile, or will not work

What you see	What it usually means
`'speed' was not declared in this scope`	Spelled differently (capitals count), made inside other { }, or a missing #include.
`expected ';' before ...`	A missing ; - usually at the end of the line ABOVE.
`expected '}' at end of input`	A { with no matching }.
`Bluepad32.h: No such file or directory`	Wrong board: pick the esp32_bluepad32 ESP32 Dev Module.
`MY_CONTROLLER has not been filled in`	Run l3a_controller_check and paste the line it prints.
The controller will not connect	MY_CONTROLLER is not this pad. Is it charged? Hold SYNC.
`Failed to connect to ESP32`	Charge-only cable, wrong port, or the vehicle is off.
Serial Monitor shows nonsense	The baud box is not 115200.
The LEDs do not change	No strip.show().
It creeps on its own	STICK_DEADZONE is too small.